Exam 2026, Part 2

Exam number 45, 46 & 47

February 06, 2028

Exercise 1: Choice of realized variance estimator

We first load the high-frequency minute price data and filter the dataset to AAPL. The timestamp column is converted to datetime format, and we define each trading day from the timestamp.

The data contain minute-level closing prices during regular trading hours. We use a fixed one-minute calendar-time grid from 09:30 to 15:59 for each trading day. This corresponds to 390 one-minute observations per normal trading day. Since a stock may not trade in every minute, we handle missing prices using previous-tick interpolation within each trading day only. This means that if AAPL has no observed price in a given minute, we carry forward the most recently observed price from the same trading day. We do not carry prices across trading days, and we do not backfill prices before the first observed price of a day.

For each sampling interval $\Delta \in \{1, 2, 5, 10, 15, 30, 60\}$, we sample prices on a fixed grid relative to the market open. For example, with 10-minute sampling, prices are taken at 09:30, 09:40, 09:50, and so on. This differs from a bucket-based approach, where the last observed price within each Δ-minute interval is used. We prefer the fixed-grid approach because it makes the sampling rule explicit and because it will later allow us to align intraday returns across all 30 stocks when constructing realized covariance matrices.

Daily realized variance is computed as:

$$RV_d^{(\Delta)} = \sum_j \left(r_{d,j}^{(\Delta)} \right)^2 ,$$

where $r_{d,j}^{(\Delta)}$ is the intraday log return over sampling interval.

As shown in Figure 1, average realized variance is highest at the shortest sampling intervals and decreases as the sampling interval increases. This pattern is consistent with market microstructure noise at very high frequencies, such as bid-ask bounce, price discreteness, and stale prices. At very fine sampling intervals, these frictions can inflate realized variance because observed transaction prices deviate from the latent efficient price. Sampling less frequently reduces this noise, but very long sampling intervals also discard useful intraday information.

The volatility signature plot shows that the average realized variance is highest at the shortest sampling intervals and falls substantially when moving from 1-minute to 5-minute sampling. This is

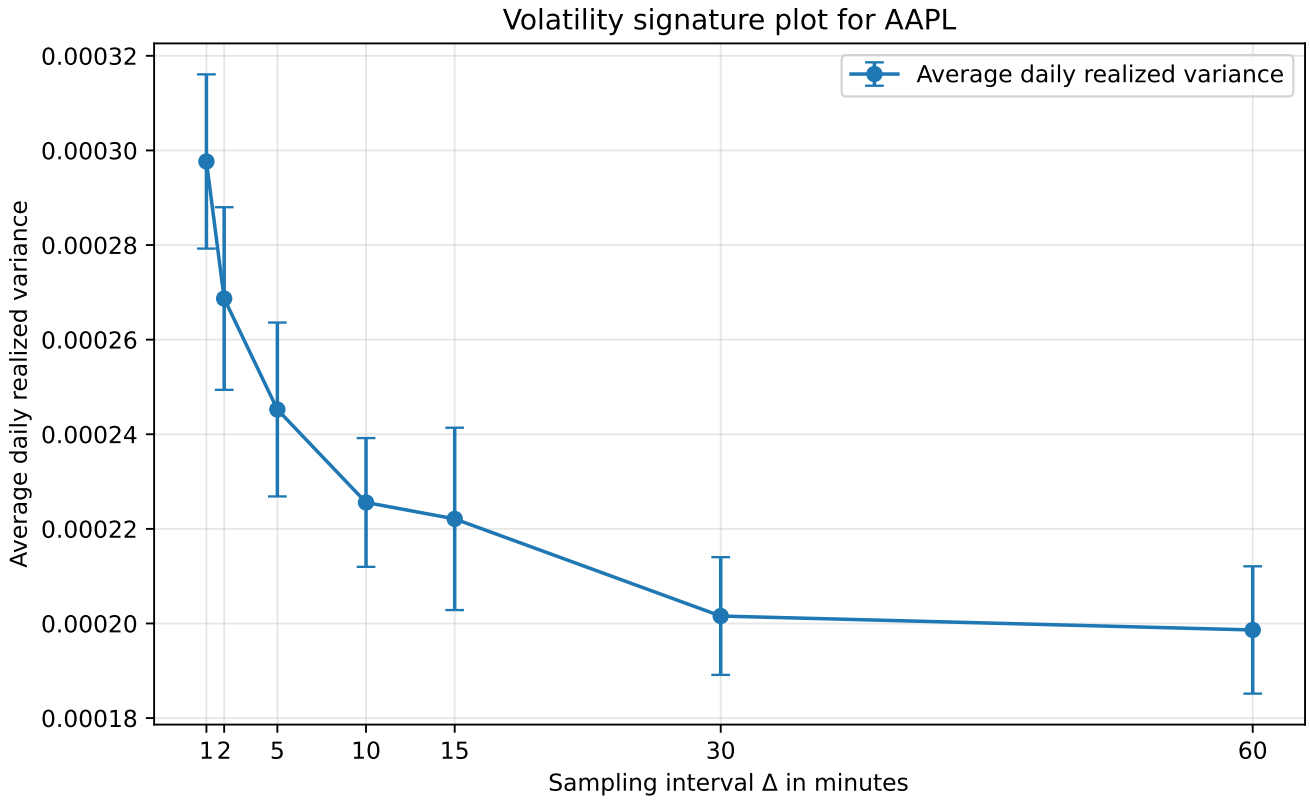

Figure 1: Volatility signature plot for AAPL with 95% confidence intervals.

consistent with the presence of market microstructure noise at very high frequencies. In the lecture slides, the observed log price is written as the latent efficient log price plus a noise component, $Y_{i\Delta_n} = X_{i\Delta_n} + U_{i\Delta_n}$, and the noise-driven fraction of return variance increases as Δ_n becomes small. Sampling less frequently therefore reduces the influence of bid-ask bounce, price discreteness and stale prices.

We choose the standard realized variance estimator based on 5-minute log returns for the remainder of the analysis:

$$RV_d^{(5)} = \sum_j \left(r_{d,j}^{(5)}\right)^2.$$

This choice is also consistent with the empirical realized-volatility literature, where 5-minute returns are commonly used as a practical compromise between using high-frequency information and limiting market microstructure noise. Andersen and Bollerslev discuss the use of high-frequency intraday returns to obtain improved volatility measurements and refer to empirical analysis based on five-minute returns. At the same time, 5-minute sampling leaves approximately $390/5 - 1 = 77$ intraday returns per trading day. Since the subsequent realized covariance matrix is 30-dimensional, this helps avoid the rank problem emphasized in the lecture slides: when the return matrix has $T < N$, the sample covariance matrix is singular and cannot be inverted. Thus, 5-minute sampling is preferred to lower-frequency choices such as 15, 30 or 60 minutes, which would leave fewer intraday returns for estimating the 30×30 covariance matrix.

Exercise 2a: Daily realized covariance

For each trading day d, we estimate a 30×30 realized covariance matrix using synchronized 5-minute log returns. This extends the realized variance estimator from Exercise 1 to the multivariate case.

Let $r_{d,j}$ denote the 30×1 vector of 5-minute log returns for all stocks at intraday interval j on trading day d. The realized covariance matrix is computed as

$$RC_d = \sum_{j=1}^{M_d} r_{d,j} r'_{d,j}.$$

The dataset is already observed at the minute level rather than at the transaction-by-transaction level. Therefore, the asynchronicity problem discussed in the lecture slides is less severe than in tick-data applications. To ensure that all assets are observed on the same time grid, prices are placed on a common one-minute calendar grid within each trading day. Missing prices are handled using previous-tick interpolation, meaning that the most recently observed price within the same trading day is carried forward. Prices are never carried across trading days.

We then sample every fifth minute and compute synchronized 5-minute log returns. This sampling frequency follows the choice from Exercise 1 and provides a compromise between reducing market microstructure noise and retaining enough intraday observations for estimating a 30×30 covariance matrix.

Exercise 2a: Daily realized covariance matrices

CHOSEN_DELTA = 5

All 30 symbols and all trading days in the dataset

symbols = np.sort(hf_prices[“symbol”].unique()) trading_days_all = np.sort(hf_prices[“trading_day”].unique())

Dictionary to store one 30 x 30 realized covariance matrix per trading day

rc_matrices = {}

for day in trading_days_all:

```
# Select prices for one trading day
day_data = (
    hf_prices[hf_prices["trading_day"] == day]
    .copy()
    .sort_values(["symbol", "ts"])
)

# Create fixed one-minute calendar grid for this trading day
market_open = market_timestamp(day, MARKET_OPEN)
market_close = market_timestamp(day, MARKET_CLOSE)

one_min_grid = pd.date_range(
    start=market_open,
    end=market_close,
    freq="1min"
)

# Convert from long format to wide format:
# rows = timestamps, columns = stocks, values = prices
prices_wide = (
    day_data
    .pivot_table(
        index="ts",
        columns="symbol",
        values="price",
        aggfunc="last"
    )
    .reindex(one_min_grid)
    .ffill()
)

# Keep columns in a fixed order
prices_wide = prices_wide[symbols]

# Sample prices every 5 minutes
prices_sampled = prices_wide.iloc[::CHOSEN_DELTA]

# Compute synchronized 5-minute log returns
returns = np.log(prices_sampled).diff().dropna()
```

```
# Compute daily realized covariance matrix:
# RC_d = sum_j r_{d,j} r_{d,j}'
rc_day = returns.T @ returns

# Store realized covariance matrix for this trading day
rc_matrices[day] = rc_day

# ------------------------------------------------------------
```

Checks for Exercise 2a

Number of realized covariance matrices

n_rc_matrices = len(rc_matrices)

Dimensions

first_day = list(rc_matrices.keys())[0] first_rc = rc_matrices[first_day]

print("Number of RC matrices:", n_rc_matrices) print("First day:", first_day) print("Shape of first RC matrix:", first_rc.shape)

Check diagonal elements for first day

print("First 5 diagonal elements on first day:") print(np.diag(first_rc)[:5])

Check total realized variance across all stocks for first day

print("Trace of first RC matrix:") print(np.trace(first_rc))

Check sum of all covariance elements for first day

print("Sum of all elements in first RC matrix:") print(first_rc.values.sum())

```
```

References